

NEET 2024 — Pandas DataFrame Practice Questions

300 practice questions converted from the supplied SQL practice set and expanded for hands-on pandas DataFrame practice.

Dataset	NEET 2024
DataFrame	df
Columns	state, city, center_name, center_num, Srlno, Marks
Skills	Selection, filtering, sorting, aggregation, groupby, feature engineering, cleaning, reshaping, merging and ranking

Instructions: Solve each question using pandas. Prefer DataFrame/Series operations such as loc, iloc, boolean indexing, sort_values, groupby, agg, transform, apply, pivot_table, crosstab, merge, rank and string methods.

Level 1: DataFrame Basics & Inspection

1. Load the NEET dataset into a pandas DataFrame named df.
2. Display the first 5 rows of the DataFrame.
3. Display the last 5 rows of the DataFrame.
4. Display the first 10 rows of the DataFrame.
5. Display the last 10 rows of the DataFrame.
6. Display the shape of the DataFrame.
7. Display the number of rows in the DataFrame.
8. Display the number of columns in the DataFrame.
9. Display all column names.
10. Display the DataFrame index.
11. Display the data types of all columns.
12. Generate a concise summary of the DataFrame using info().
13. Generate descriptive statistics for the Marks column.
14. Generate descriptive statistics for all numeric columns.
15. Generate descriptive statistics for all columns, including non-numeric columns.
16. Check whether the DataFrame contains any missing values.
17. Count missing values in each column.
18. Check whether the DataFrame contains duplicate rows.
19. Count the total number of duplicate rows.
20. Display the memory usage of the DataFrame.

Level 2: Selecting Rows and Columns

21. Display only the state column.
22. Display only the city column.
23. Display only the Marks column.
24. Display state, city, and Marks.
25. Display center_name and center_num.

26. Display the first 20 rows using `iloc`.
27. Display rows 10 to 19 using `iloc`.
28. Display rows 50 to 59 using `iloc`.
29. Display the first 10 rows and first 3 columns using `iloc`.
30. Display rows 5 to 15 and columns state through Marks using `iloc`.
31. Select the row with index 0 using `loc`.
32. Select rows with indexes 0, 5, and 10 using `loc`.
33. Select state and Marks for rows with indexes 0 through 9.
34. Set `Srln` as the DataFrame index.
35. Reset the DataFrame index after setting `Srln` as the index.
36. Select all columns except Marks.
37. Select all columns except state.
38. Select state, city, center_name, and Marks.
39. Create a new DataFrame containing only city and Marks.
40. Create a copy of `df` containing state, city, and Marks.

Level 3: Filtering with Boolean Conditions

41. Display students who scored more than 650 marks.
42. Display students who scored less than 200 marks.
43. Display students who scored between 500 and 600 marks.
44. Display students who scored exactly 720 marks.
45. Display students who scored 0 marks.
46. Display students whose Marks are greater than or equal to 600.
47. Display students whose Marks are less than or equal to 300.
48. Display students whose Marks are not equal to 0.
49. Display students from a particular state.
50. Display students from a particular city.
51. Display students from a particular examination centre.
52. Display students not belonging to a particular state.
53. Display students whose Marks are 600, 610, or 620.
54. Display students whose city starts with the letter A.
55. Display students whose city ends with the letter A.
56. Display students whose city contains the word 'pur'.
57. Display students whose center_name contains 'Public School'.
58. Display students whose center_name contains the word 'School'.
59. Display students whose state starts with a specific letter.
60. Display students whose state name contains a specific substring.

Level 4: Sorting, Ranking & Top/Bottom Records

61. Sort students by Marks in ascending order.
62. Sort students by Marks in descending order.
63. Sort students by state alphabetically.
64. Sort students by city alphabetically.
65. Sort students by center_name alphabetically.

66. Sort by state and then Marks.
67. Sort by city and Marks in descending order.
68. Sort by state ascending and Marks descending.
69. Sort by center_name ascending and Marks descending.
70. Display the top 20 scorers.
71. Display the bottom 20 scorers.
72. Display the top 50 scorers.
73. Display the bottom 50 scorers.
74. Display the highest-scoring student.
75. Display the lowest-scoring student.
76. Display the second-highest scorer.
77. Display the second-lowest scorer.
78. Display the top 10 scorers with only state, city, and Marks.
79. Display the top 10 scorers from a selected state.
80. Display the top 10 scorers from a selected city.

Level 5: Aggregations & Descriptive Statistics

81. Find the maximum Marks.
82. Find the minimum Marks.
83. Find the average Marks.
84. Find the median Marks.
85. Find the standard deviation of Marks.
86. Find the variance of Marks.
87. Find the total Marks scored by all students.
88. Find the range between maximum and minimum Marks.
89. Find the 25th percentile of Marks.
90. Find the 50th percentile of Marks.
91. Find the 75th percentile of Marks.
92. Find the 90th percentile of Marks.
93. Find the 95th percentile of Marks.
94. Find the number of students.
95. Count the number of non-null Marks values.
96. Count students scoring above 600.
97. Count students scoring below 300.
98. Count students scoring between 500 and 600.
99. Count students scoring exactly 720.
100. Count students scoring exactly 0.

Level 6: value_counts, nunique & Frequency Analysis

101. Count the number of students in each state using value_counts().
102. Count the number of students in each city.
103. Count the number of students in each centre.
104. Display the top 10 states by number of students.
105. Display the top 10 cities by number of students.

- 106. Display the top 10 centres by number of students.
- 107. Display the bottom 10 states by number of students.
- 108. Display the bottom 10 cities by number of students.
- 109. Display the bottom 10 centres by number of students.
- 110. Find the number of unique states.
- 111. Find the number of unique cities.
- 112. Find the number of unique examination centres.
- 113. Find the most common state.
- 114. Find the most common city.
- 115. Find the most common examination centre.
- 116. Find the least common state.
- 117. Find the least common city.
- 118. Find the least common examination centre.
- 119. Create a frequency table of Marks.
- 120. Create a frequency table of state and city combinations.

Level 7: groupby Operations

- 121. Find the number of students in each state using groupby().
- 122. Find the number of students in each city using groupby().
- 123. Find the number of students in each centre using groupby().
- 124. Find the average Marks by state.
- 125. Find the average Marks by city.
- 126. Find the average Marks by examination centre.
- 127. Find the maximum Marks by state.
- 128. Find the minimum Marks by state.
- 129. Find the maximum Marks by city.
- 130. Find the minimum Marks by city.
- 131. Find the maximum Marks by centre.
- 132. Find the minimum Marks by centre.
- 133. Find the total Marks by state.
- 134. Find the total Marks by city.
- 135. Find the total Marks by centre.
- 136. Find the median Marks by state.
- 137. Find the median Marks by city.
- 138. Find the standard deviation of Marks by state.
- 139. Find the standard deviation of Marks by city.
- 140. Find the number of 650+ scorers in each state.

Level 8: agg, transform, filter & apply

- 141. Use groupby().agg() to calculate count, mean, maximum, and minimum Marks by state.
- 142. Use groupby().agg() to calculate count, mean, and standard deviation by city.
- 143. Use groupby().agg() to calculate total and average Marks by centre.
- 144. Give meaningful names to aggregated columns for state-level statistics.
- 145. Calculate multiple aggregations for Marks by state in one operation.

- 146. Calculate the number of students and average Marks for each city.
- 147. Calculate the number of students, total Marks, and average Marks for each centre.
- 148. Filter states having more than 10,000 students using `groupby().filter()`.
- 149. Filter cities having more than 5,000 students using `groupby().filter()`.
- 150. Filter centres having more than 300 students using `groupby().filter()`.
- 151. Filter states whose average Marks exceed 450.
- 152. Filter cities whose average Marks exceed 500.
- 153. Filter centres whose average Marks exceed 550.
- 154. Filter states where the maximum Marks exceed 700.
- 155. Filter cities having at least 20 students scoring above 650.
- 156. Use `transform()` to add each student's state average Marks.
- 157. Use `transform()` to add each student's city average Marks.
- 158. Use `transform()` to add each student's centre average Marks.
- 159. Use `transform()` to add the number of students in each state to every row.
- 160. Use `transform()` to calculate each student's percentage of their state's total Marks.

Level 9: New Columns & Feature Engineering

- 161. Create a column `Pass_Fail` where students scoring at least 300 are Pass, otherwise Fail.
- 162. Create a `Grade` column with A/B/C/D based on Marks.
- 163. Create score bands such as 0–100, 101–200, 201–300, and so on.
- 164. Create a `Score_Band` column using `pd.cut()`.
- 165. Count students in each `Score_Band`.
- 166. Create a `High_Medium_Low` column based on Marks.
- 167. Create a `Scholarship_Eligible` column based on a chosen Marks threshold.
- 168. Create a `Performance_Category` column with at least four categories.
- 169. Calculate the numeric percentage equivalent assuming Marks are out of 720.
- 170. Create a column indicating whether a student scored 600 or more.
- 171. Create a column indicating whether a student scored 650 or more.
- 172. Create a column indicating whether a student scored 700 or more.
- 173. Create a column indicating whether a student scored exactly 720.
- 174. Create a column indicating whether a student scored below 200.
- 175. Create a column containing the length of each city name.
- 176. Create a column containing the length of each centre name.
- 177. Create a column containing the first letter of each city.
- 178. Create a column containing the first letter of each state.
- 179. Create a column combining state and city into one location label.
- 180. Create a column combining city and centre name into one label.

Level 10: String Operations & Cleaning

- 181. Convert state names to uppercase.
- 182. Convert city names to lowercase.
- 183. Remove leading and trailing spaces from state values.
- 184. Remove leading and trailing spaces from city values.
- 185. Find rows where city contains a specific word, ignoring case.

- 186. Find rows where center_name contains 'school', ignoring case.
- 187. Extract the first word from center_name.
- 188. Extract the last word from center_name.
- 189. Find the length of each state name.
- 190. Find the length of each city name.
- 191. Find the length of each centre name.
- 192. Replace a selected word in center_name with another word.
- 193. Replace multiple variations of a centre name with a standard value.
- 194. Check whether city values start with a particular prefix.
- 195. Check whether city values end with a particular suffix.
- 196. Split center_name into separate words.
- 197. Extract a specific part of center_name using string slicing.
- 198. Find the number of spaces in each center_name.
- 199. Create a standardized version of state and city names.
- 200. Find duplicate centre names after converting them to lowercase.

Level 11: Missing Values, Duplicates & Data Quality

- 201. Count missing values in every column.
- 202. Display rows containing at least one missing value.
- 203. Display rows where Marks is missing.
- 204. Display rows where state is missing.
- 205. Display rows where city is missing.
- 206. Fill missing Marks with the median Marks.
- 207. Fill missing Marks with the mean Marks.
- 208. Fill missing categorical values with 'Unknown'.
- 209. Drop rows where Marks is missing.
- 210. Drop rows where state or city is missing.
- 211. Check for duplicated Srln values.
- 212. Display all duplicated Srln records.
- 213. Remove duplicate rows while keeping the first occurrence.
- 214. Remove duplicate rows while keeping the last occurrence.
- 215. Check whether center_num is unique within center_name.
- 216. Check whether every row has a valid Marks value between 0 and 720.
- 217. Display rows with Marks outside the valid 0–720 range.
- 218. Count invalid Marks values outside the range 0–720.
- 219. Check for blank strings in state, city, and center_name.
- 220. Replace blank strings with missing values.

Level 12: Advanced Filtering & Comparative Analysis

- 221. Find students scoring above the overall average Marks.
- 222. Find students scoring below the overall average Marks.
- 223. Find students scoring equal to the maximum Marks.
- 224. Find students scoring equal to the minimum Marks.
- 225. Find states whose average Marks exceed the national average.

- 226. Find cities whose average Marks exceed the overall average.
- 227. Find centres whose average Marks exceed the overall average.
- 228. Find students whose Marks exceed their state average.
- 229. Find students whose Marks exceed their city average.
- 230. Find students whose Marks exceed their centre average.
- 231. Find the state with the highest average Marks.
- 232. Find the state with the lowest average Marks.
- 233. Find the city with the highest average Marks.
- 234. Find the city with the lowest average Marks.
- 235. Find the centre with the highest average Marks.
- 236. Find the centre with the lowest average Marks.
- 237. Find the state with the largest number of 650+ scorers.
- 238. Find the city with the largest number of 650+ scorers.
- 239. Find the centre with the largest number of 650+ scorers.
- 240. Find centres with more than the average number of students.

Level 13: Pivot Tables, Crosstabs & MultiIndex

- 241. Create a pivot table showing average Marks by state and city.
- 242. Create a pivot table showing student count by state and city.
- 243. Create a pivot table showing total Marks by state and city.
- 244. Create a pivot table showing maximum Marks by state and city.
- 245. Create a pivot table showing minimum Marks by state and city.
- 246. Create a pivot table showing average Marks by state and centre.
- 247. Create a pivot table showing student counts by city and centre.
- 248. Create a pivot table of Score_Band against state.
- 249. Create a crosstab of state and score categories.
- 250. Create a crosstab of city and Pass_Fail.
- 251. Create a crosstab of state and Pass_Fail.
- 252. Create a crosstab of state and High_Medium_Low.
- 253. Create a pivot table with state as rows and Score_Band as columns.
- 254. Create a pivot table with city as rows and Score_Band as columns.
- 255. Add margins/totals to a suitable pivot table.
- 256. Normalize a state-by-score-band crosstab to percentages by row.
- 257. Normalize a state-by-score-band crosstab to percentages by column.
- 258. Create a MultiIndex DataFrame using state and city.
- 259. Select data from a MultiIndex using .loc.
- 260. Sort a MultiIndex by state and city.

Level 14: Merging, Joining & Relational-Style Operations

- 261. Create a state-level summary DataFrame containing student count and average Marks.
- 262. Create a city-level summary DataFrame containing student count and average Marks.
- 263. Merge the state-level summary back into the original DataFrame.
- 264. Merge the city-level summary back into the original DataFrame.
- 265. Merge a centre-level summary back into the original DataFrame.

- 266. Join state-level statistics to each student record.
- 267. Join city-level statistics to each student record.
- 268. Create separate DataFrames for state totals and state averages, then merge them.
- 269. Create separate DataFrames for city totals and city averages, then merge them.
- 270. Merge two summaries using state as the key.
- 271. Merge two summaries using city as the key.
- 272. Create a centre-to-city mapping DataFrame and join it to student data.
- 273. Identify records that do not match during a merge using an indicator column.
- 274. Perform a left merge between student data and state statistics.
- 275. Perform an inner merge between student data and city statistics.
- 276. Perform an outer merge between two summary DataFrames.
- 277. Use merge validation to check whether a state-level merge is one-to-many.
- 278. Merge state and city summaries to create a combined location report.
- 279. Compare two summary DataFrames and identify states with changed averages.
- 280. Create a table containing each state, its average Marks, and national average.

Level 15: Window-Style Analysis, Ranking & Advanced Challenges

- 281. Calculate the overall rank of every student using rank().
- 282. Calculate dense rank of students based on Marks.
- 283. Calculate competition rank of students based on Marks.
- 284. Calculate percentile rank of every student.
- 285. Calculate rank within each state.
- 286. Calculate rank within each city.
- 287. Calculate rank within each centre.
- 288. Identify the top scorer in every state.
- 289. Identify the top scorer in every city.
- 290. Identify the top scorer in every centre.
- 291. Identify the top 3 scorers in every state.
- 292. Identify the top 3 scorers in every city.
- 293. Identify the top 3 scorers in every centre.
- 294. Find the difference between each student's Marks and the highest Marks in their state.
- 295. Find the difference between each student's Marks and the average Marks of their state.
- 296. Find the percentage of the state maximum achieved by each student.
- 297. Find the percentage of the centre maximum achieved by each student.
- 298. Calculate each state's contribution to total candidates.
- 299. Calculate each city's contribution to total candidates.
- 300. Calculate each centre's contribution to total candidates.

End of Practice Set — 300 Questions